ПРАКТИЧЕСКАЯ РАБОТА №1

Тема: «Сравнение архитектуры операционных систем Windows NT, Linux и MINIX 3»

Дисциплина: …

Преподаватель: Семак Вячеслав Павлович

Выполнил: …

Группа: …

Дата выполнения: «__»_____ 20__ г.

2026

Оглавление

Введение.....3
Цель и задачи работы.....4
Средства выполнения работы.....6
Ход работы.....7
1. Рассматриваемые версии операционных систем.....7
2. Архитектура Windows NT.....8
3. Архитектура ОС на базе Linux.....10
4. Архитектура MINIX 3.....12
5. Сравнительный анализ архитектур.....14
6. Влияние размещения компонентов на изоляцию, устойчивость и безопасность.....19
7. Разбор ситуации компрометации драйвера.....21
Вывод.....24
Список использованных источников.....25

Введение

Операционная система (ОС) представляет собой комплекс программ, управляющих аппаратными ресурсами компьютера и предоставляющих прикладным программам единый интерфейс для работы с процессором, памятью, устройствами и файлами. Центральным элементом любой ОС является ядро – программный код, выполняющийся в привилегированном режиме процессора и имеющий неограниченный доступ к оборудованию.

Архитектура ОС определяет, какие компоненты выполняются в пространстве ядра, а какие вынесены в пространство пользователя, и каким образом эти компоненты взаимодействуют. По этому признаку выделяют три основных типа ядер: монолитные, в которых все основные службы работают в едином адресном пространстве ядра; микроядра, в которых в режиме ядра оставлен минимальный набор механизмов, а драйверы и системные службы работают как обычные процессы; гибридные ядра, сочетающие черты обоих подходов.

Выбор архитектуры напрямую влияет на производительность, устойчивость к отказам отдельных компонентов и информационную безопасность системы. Ошибка или уязвимость в компоненте, выполняющемся в режиме ядра, способна нарушить работу всей ОС, тогда как сбой изолированного компонента в пространстве пользователя затрагивает только его самого.

В качестве объектов исследования в работе рассматриваются три ОС, представляющие разные архитектурные подходы: Windows NT – семейство ОС корпорации Microsoft с гибридным ядром, Linux – свободное монолитное модульное ядро, лежащее в основе большого числа дистрибутивов, и MINIX 3 – исследовательская ОС на основе микроядра, разработанная под руководством Э. Таненбаума с акцентом на надёжность и самовосстановление.

Цель и задачи работы

Цель работы:

Получение практических и теоретических навыков в сравнительном анализе архитектуры операционных систем. В данной практической работе необходимо произвести сравнение архитектуры Windows NT, Linux и MINIX 3, определить расположение основных системных компонентов и проанализировать значение архитектурных решений для изоляции, устойчивости и информационной безопасности.

Задачи для выполнения работы:

1. Изучить техническую документацию по архитектуре Windows NT, Linux и MINIX 3, указав рассматриваемые версии систем и использованные источники.
2. Определить тип ядра каждой операционной системы и выделить особенности её архитектурной организации.
3. Установить назначение и расположение основных компонентов ОС: ядра, системных служб, драйверов, компонентов файловой системы, средств управления процессами и памятью.
4. Рассмотреть способы взаимодействия приложений, системных служб, драйверов и ядра, обозначив границы пространства пользователя и пространства ядра.
5. Составить упрощённые архитектурные схемы Windows NT, Linux и MINIX 3, отразив размещение компонентов и связи между ними.
6. Выполнить сравнительный анализ трёх ОС по единым критериям и представить результаты в сравнительной таблице.
7. Проанализировать влияние размещения и полномочий системных компонентов на их изоляцию, устойчивость ОС и информационную безопасность.

8. Рассмотреть ситуацию компрометации драйвера, определить возможные последствия для конфиденциальности, целостности и доступности и предложить меры снижения риска.

9. Сформулировать обоснованные выводы о сходствах, различиях и ограничениях исследованных архитектур.

10. Оформить отчёт, содержащий архитектурные схемы, сравнительную таблицу, разбор ситуации, выводы и список источников.

Средства выполнения работы

Для выполнения практической работы были использованы технические, программные и информационные средства, перечисленные ниже.

1. Технические средства:

- персональный компьютер с установленной операционной системой;
- доступ к сети Интернет для поиска и изучения технической документации.

2. Программные средства:

- браузер – для поиска, просмотра документации и сохранения ссылок на источники;
- текстовый редактор – для подготовки отчёта, заполнения сравнительной таблицы и оформления выводов;
- графический редактор – для построения архитектурных схем;
- средство просмотра PDF – для работы с документацией и проверки итогового отчёта.

3. Информационные средства:

- официальная техническая документация Microsoft по архитектуре Windows NT и организации системных компонентов [2–5];
- документация ядра Linux и дистрибутива Ubuntu [6–9];
- официальная документация и публикации проекта MINIX 3 [10–12];
- учебная литература по архитектуре операционных систем [1, 13].

Ход работы

1. Рассматриваемые версии операционных систем

На первом этапе работы были выбраны конкретные версии исследуемых ОС, поскольку архитектура каждой системы развивается: компоненты переносятся между пространством ядра и пространством пользователя, появляются новые механизмы изоляции. Для анализа были выбраны актуальные на момент выполнения работы версии:

- Windows NT – Windows 11 версии 24H2 (ядро NT 10.0, сборка 26100) для архитектуры x86-64; основные источники – книга «Windows Internals» [1] и документация Microsoft Learn [2–5];
- ОС на базе Linux – дистрибутив Ubuntu 24.04 LTS «Noble Numbat» с ядром Linux 6.8 для архитектуры x86-64; основные источники – официальная документация ядра [6–8] и примечания к выпуску дистрибутива [9];
- MINIX 3 – версия 3.4.0 (выпуск-кандидат rc6) для архитектуры x86; основные источники – книга Э. Таненбаума и А. Вудхалла [10], публикация разработчиков о самовосстанавливающейся архитектуре [11] и материалы официального сайта проекта [12].

Для сопоставимости результатов все три системы были рассмотрены по единому набору критериев: тип ядра, состав пространства ядра и пространства пользователя, способы взаимодействия компонентов, размещение драйверов и файловых служб, организация управления процессами и памятью, механизмы изоляции и восстановления, а также особенности, значимые для безопасности.

2. Архитектура Windows NT

Ядро Windows NT относится к гибридному типу. Изначально архитектура NT проектировалась по образцу микроядерных систем: небольшое ядро (Kernel), над которым располагается исполнительная система (Executive), и подсистемы окружения, работающие в пространстве пользователя. Однако для повышения производительности большая часть системных служб, драйверы и, начиная с Windows NT 4.0, оконный менеджер и графический интерфейс GDI (Win32k.sys) были размещены в режиме ядра в едином адресном пространстве [1, 2].

В режиме ядра выполняются: уровень аппаратных абстракций HAL (hal.dll), скрывающий различия аппаратных платформ; ядро, отвечающее за планирование потоков, обработку прерываний и исключений, синхронизацию; исполнительная система в файле ntoskrnl.exe, включающая диспетчер объектов, диспетчер ввода-вывода, диспетчер памяти, диспетчер процессов и потоков, диспетчер кэша, диспетчеры Plug and Play и электропитания, монитор безопасности (Security Reference Monitor), диспетчер конфигурации (реестр) и механизм локального вызова процедур ALPC; драйверы устройств и файловых систем (*.sys) [1, 2].

В пространстве пользователя размещаются приложения, системные процессы (диспетчер сеансов smss.exe, процесс подсистемы Win32 csrss.exe, wininit.exe, winlogon.exe, процесс локальной проверки подлинности lsass.exe), процессы служб (диспетчер служб services.exe и хост-процессы svchost.exe), библиотеки подсистем (kernel32.dll, user32.dll, gdi32.dll и др.), а также библиотека ntdll.dll, содержащая заглушки системных вызовов. В пространстве пользователя также могут работать драйверы модели UMDF, загружаемые в хост-процесс WUDFHost.exe [3].

Взаимодействие компонентов построено следующим образом: приложение вызывает функцию Win32 API, библиотека подсистемы обращается к функции Native API в ntdll.dll, которая выполняет инструкцию

syscall и передаёт управление диспетчеру системных служб ядра. Операции ввода-вывода оформляются диспетчером ввода-вывода в виде пакетов IRP, проходящих по стеку драйверов, а процессы пространства пользователя обмениваются данными через ALPC и RPC.

Размещение компонентов Windows NT и границы режимов процессора показаны на схеме (Рисунок 1 – Упрощённая архитектурная схема Windows NT (Windows 11, версия 24H2)).

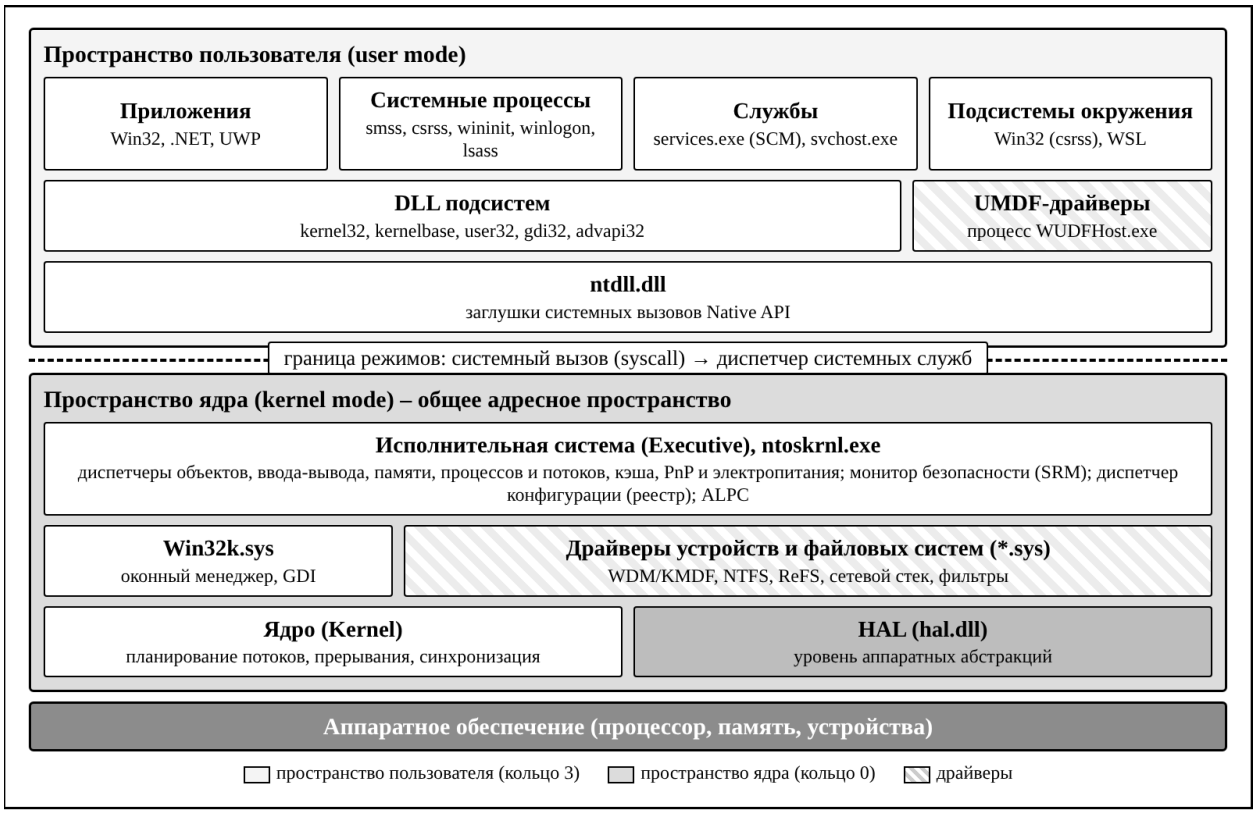


Рисунок 1 – Упрощённая архитектурная схема Windows NT (Windows 11, версия 24H2)

Из схемы следует, что граница между пространством пользователя и пространством ядра в Windows NT проходит ниже библиотеки ntdll.dll, а все компоненты исполнительной системы, драйверы и графическая подсистема разделяют общее адресное пространство ядра и не изолированы друг от друга. Дополнительный уровень изоляции в современных версиях обеспечивается безопасностью на основе виртуализации (VBS): гипервизор Hyper-V выделяет защищённую среду VTL1, в которой работают Secure Kernel и изолированный процесс LsaIso.exe, недоступные даже для кода режима ядра основной системы [1, 4].

3. Архитектура ОС на базе Linux

Ядро Linux является монолитным: планировщик, подсистема управления памятью, виртуальная файловая система (VFS), файловые системы, сетевой стек, механизмы межпроцессного взаимодействия и драйверы устройств выполняются в едином адресном пространстве ядра и вызывают функции друг друга напрямую. Монолитность сочетается с модульностью: драйверы и файловые системы могут быть собраны в виде загружаемых модулей (*.ko), которые подключаются к работающему ядру без перезагрузки, но после загрузки получают те же полномочия, что и остальной код ядра [6].

В ядре 6.8, используемом в Ubuntu 24.04 LTS, процессы и потоки представлены единой структурой задачи, а распределение процессорного времени выполняет планировщик EEVDF, заменивший CFS начиная с версии 6.6. Подсистема памяти реализует страничную виртуальную память с копированием при записи, распределитель страниц buddy и распределитель объектов SLUB, кэш страниц файловой системы. Для ограничения и изоляции групп процессов используются контрольные группы (cgroups) и пространства имён (namespaces), а модули безопасности LSM (AppArmor в Ubuntu, SELinux в других дистрибутивах) реализуют мандатное управление доступом [6, 9].

В пространстве пользователя работают приложения, стандартная библиотека C (glibc), система инициализации и управления службами systemd, системные службы (journald, udevd, sshd, NetworkManager), командная оболочка и утилиты. Отдельные драйверы и файловые системы могут выполняться в пространстве пользователя с помощью интерфейсов FUSE, UIO и VFIO, а также библиотеки libusb.

Взаимодействие пространства пользователя с ядром осуществляется через интерфейс системных вызовов, виртуальные файловые системы /proc и /sys, вызовы ioctl к файлам устройств и сокеты netlink (Рисунок 2 – Упрощённая архитектурная схема ОС на базе Linux (Ubuntu 24.04 LTS, ядро 6.8)).

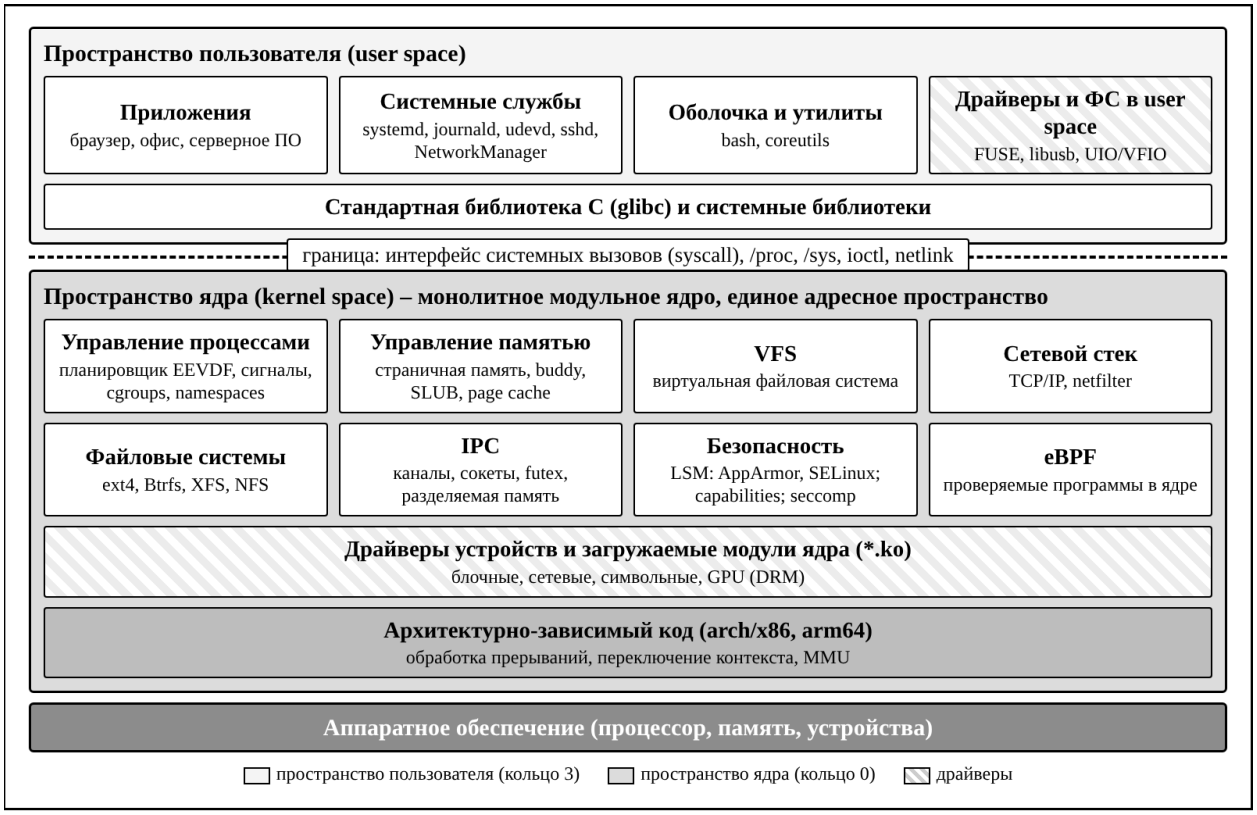


Рисунок 2 – Упрощённая архитектурная схема ОС на базе Linux (Ubuntu 24.04 LTS, ядро 6.8)

Схема показывает, что граница привилегий в Linux проходит по интерфейсу системных вызовов: всё, что расположено ниже неё, включая сторонние драйверы и модули, выполняется в нулевом кольце защиты и имеет доступ ко всей памяти ядра. Механизм eBPF позволяет загружать в ядро небольшие программы, однако перед выполнением они проходят проверку верификатором, что ограничивает их возможности по сравнению с обычными модулями [6].

4. Архитектура MINIX 3

MINIX 3 построена на основе микроядра и многосерверной организации. В режиме ядра выполняется только микроядро объёмом порядка нескольких тысяч строк кода, которое обрабатывает прерывания, переключает процессы, настраивает блок управления памятью и передаёт сообщения между процессами. В адресном пространстве ядра также работают две задачи: SYSTEM, обслуживающая вызовы ядра от привилегированных процессов, и CLOCK, отвечающая за таймеры и учёт времени [10, 11].

Все остальные компоненты вынесены в пространство пользователя и организованы в уровни. На втором уровне работают драйверы устройств (диска, терминала, сетевых адаптеров, аудио и др.), каждый из которых является отдельным процессом без привилегий нулевого кольца. На третьем уровне располагаются системные серверы: менеджер процессов PM, сервер виртуальной памяти VM, сервер политики планирования SCHED, виртуальная файловая система VFS, серверы файловых систем MFS и ext2, сетевой сервер INET, хранилище данных DS и сервер реинкарнации RS. Четвёртый уровень составляют пользовательские процессы, начиная с версии 3.2 использующие библиотеки и утилиты NetBSD [10, 12].

Взаимодействие компонентов выполняется исключительно обменом сообщениями через ядро с помощью примитивов send, receive, sendrec, notify и senda. Системный вызов POSIX в пользовательском процессе преобразуется библиотекой в сообщение соответствующему серверу, например PM или VFS, который при необходимости обращается к драйверу, а тот – к ядру.

Права каждого процесса задаются маской привилегий: перечнем разрешённых вызовов ядра, адресатов сообщений, портов ввода-вывода и линий прерываний; доступ к памяти другого процесса возможен только через явно выданные гранты памяти. Размещение компонентов MINIX 3 по уровням показано на схеме (Рисунок 3 – Упрощённая архитектурная схема MINIX 3 (версия 3.4.0)).

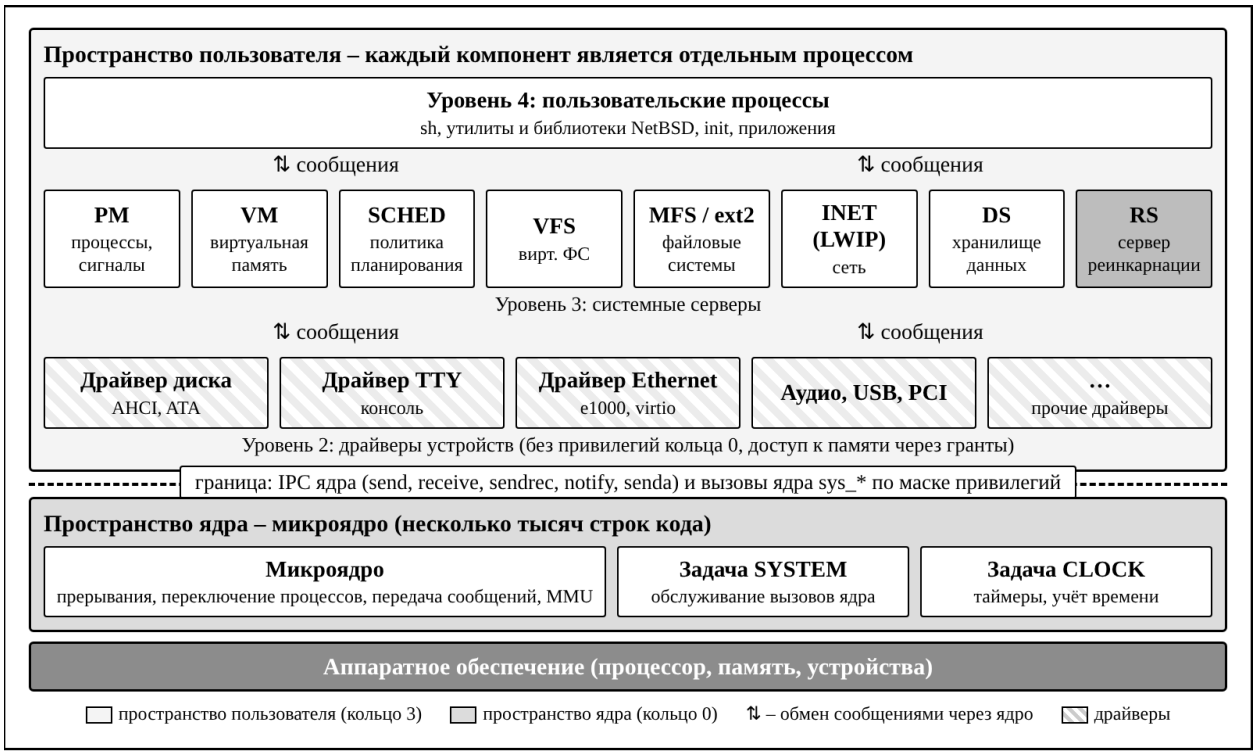


Рисунок 3 – Упрощённая архитектурная схема MINIX 3 (версия 3.4.0)

Особое место в архитектуре занимает сервер реинкарнации RS. Он запускает драйверы и серверы, периодически опрашивает их и при аварийном завершении, зависании или по команде администратора автоматически перезапускает отказавший компонент. Сохранение необходимого состояния обеспечивается сервером DS. Для драйверов блочных устройств восстановление происходит прозрачно для пользовательских процессов: файловый сервер повторяет незавершённые операции после перезапуска драйвера [11].

5. Сравнительный анализ архитектур

По результатам изучения документации и построения архитектурных схем был выполнен сравнительный анализ трёх операционных систем по единым критериям. Результаты сравнения сведены в таблицу (Таблица 1 – Сравнительная таблица операционных систем).

Таблица 1 – Сравнительная таблица операционных систем

Критерий	Windows NT	ОС на базе Linux	MINIX 3
Рассматриваемая версия ОС	Windows 11 версии 24H2 (NT 10.0, сборка 26100), x86-64	Ubuntu 24.04 LTS «Noble Numbat», ядро Linux 6.8, x86-64	MINIX 3.4.0 (rc6), x86
Название и тип ядра	ntoskrnl.exe; гибридное ядро: микроядерная структура с размещением исполнительной системы, драйверов и графической подсистемы в режиме ядра	Linux (vmlinuz); монолитное ядро с поддержкой загружаемых модулей	Микроядро MINIX 3; многосерверная микроядерная архитектура
Основные компоненты пространства ядра	HAL; ядро (планирование потоков, прерывания, синхронизация); исполнительная система (диспетчеры объектов, ввода-вывода, памяти, процессов, кэша, PnP, монитор безопасности, реестр, ALPC); Win32k.sys; драйверы устройств и файловых систем	Планировщик процессов; подсистема управления памятью; VFS и файловые системы; сетевой стек; IPC; LSM; eBPF; драйверы и модули ядра; архитектурно-зависимый код	Микроядро (прерывания, переключение процессов, передача сообщений, MMU); задачи SYSTEM и CLOCK

Продолжение Таблицы 1

Критерий	Windows NT	ОС на базе Linux	MINIX 3
Основные компоненты пространства пользователя	Приложения; системные процессы (smss, csrss, wininit, winlogon, lsass); службы (services.exe, svchost.exe); подсистемы окружения; DLL подсистем; ntdll.dll; UMDF-драйверы	Приложения; glibc; systemd и системные службы; оболочка и утилиты; графическая подсистема; FUSE-ФС и драйверы пространства пользователя	Пользовательские процессы; серверы PM, VM, SCHED, VFS, MFS, INET, DS, RS; все драйверы устройств
Взаимодействие приложений, служб и ядра	Win32 API → ntdll.dll → инструкция syscall → диспетчер системных служб; ALPC и RPC между процессами; пакеты IRP в стеке драйверов	Системные вызовы через glibc; /proc, /sys, ioctl, netlink, сокеты; компоненты ядра вызывают функции друг друга напрямую	Обмен сообщениями через ядро (send, receive, sendrec, notify); POSIX-вызовы преобразуются в сообщения серверам PM и VFS; вызовы ядра sys_* по маске привилегий
Размещение драйверов устройств	Преимущественно в режиме ядра (WDM, KMDF); часть драйверов – в режиме пользователя (UMDF, процесс WUDFHost.exe)	В пространстве ядра (встроенные или загружаемые модули *.ko); отдельные классы – в пространстве пользователя (FUSE, libusb, UIO, VFIO)	В пространстве пользователя; каждый драйвер – отдельный процесс с ограниченными привилегиями
Размещение файловых служб	Драйверы ФС (ntfs.sys, refs.sys, fastfat.sys) и драйверы-фильтры в режиме ядра; диспетчер кэша в исполнительной системе	VFS и драйверы ФС (ext4, Btrfs, XFS) в ядре; файловые системы FUSE – в пространстве пользователя	Сервер VFS и серверы ФС (MFS, ext2, ISOFS) – отдельные процессы пространства пользователя

Продолжение Таблицы 1

Критерий	Windows NT	ОС на базе Linux	MINIX 3
Управление процессами и памятью – основные особенности	Процессы и потоки – объекты исполнительной системы; вытесняющее приоритетное планирование потоков (32 уровня приоритета); страничная виртуальная память, рабочие наборы, файл подкачки	Процессы и потоки – задачи ядра; планировщик EEVDF, cgroups, namespaces; страничная виртуальная память, копирование при записи, кэш страниц, OOM killer	Механизм в ядре, политика в серверах: PM управляет процессами и сигналами, SCHED – политикой планирования, VM – выделением и отображением памяти
Способы изоляции системных компонентов	Режимы процессора (кольца 0 и 3); отдельные адресные пространства процессов; уровни целостности, защищённые процессы (PPL); VBS: изоляция Secure Kernel и LsaIso гипервизором. Компоненты режима ядра между собой не изолированы	Кольца 0 и 3; отдельные адресные пространства процессов; namespaces, cgroups, seccomp, LSM для процессов. Модули и подсистемы ядра между собой не изолированы	Отдельное адресное пространство для каждого сервера и драйвера; маски привилегий (вызовы ядра, адресаты IPC, порты ввода-вывода, IRQ); доступ к чужой памяти только через гранты
Возможные последствия сбоя драйвера	Сбой драйвера режима ядра приводит к критической ошибке (bugcheck, «синий экран») и перезагрузке; сбой UMDF-драйвера ограничен хост-процессом	Oops с завершением текущего контекста или kernel panic; возможна порча структур ядра и данных; ядро помечается как «tainted»	Аварийно завершается только процесс драйвера; остальные компоненты продолжают работу

Продолжение Таблицы 1

Критерий	Windows NT	ОС на базе Linux	MINIX 3
Средства восстановления после отказа компонента	Автоматическая перезагрузка и дампы памяти; среда восстановления WinRE, безопасный режим; TDR для видеодрайвера; параметры восстановления служб в SCM; перезапуск хоста UMDF	kdump и kexec; параметр panic=N для автоматической перезагрузки; сторожевой таймер; перезапуск служб systemd; livepatch	Сервер реинкарнации RS обнаруживает отказ и перезапускает драйвер или сервер; DS хранит состояние; для блочных драйверов восстановление прозрачно для приложений
Архитектурные особенности, значимые для безопасности	Обязательная цифровая подпись драйверов; PatchGuard; целостность памяти (HVCI); список блокировки уязвимых драйверов; KASLR, SMEP; большая доверенная вычислительная база (TCB)	Подпись модулей и режим lockdown при Secure Boot; KASLR, SMEP, SMAP; LSM, seccomp; большая TCB (более 30 млн строк в дереве исходного кода ядра)	Минимальная TCB; принцип наименьших привилегий; ограничения – накладные расходы IPC, меньшая зрелость и меньший набор современных средств защиты

Конец Таблицы 1

Сравнение показывает, что во всех трёх системах используется аппаратное разделение режимов процессора и изоляция адресных пространств пользовательских процессов, а интерфейс для приложений во всех случаях близок к модели системных вызовов. Различия касаются прежде всего того, какая часть системы выполняется с максимальными привилегиями: в Windows NT и Linux в пространстве ядра находятся драйверы, файловые системы и сетевой стек, тогда как в MINIX 3 они вынесены в отдельные непривилегированные процессы.

Windows NT по формальной структуре наиболее близка к микроядерной модели (выделены ядро, HAL и исполнительная система, присутствуют подсистемы окружения и механизм ALPC), однако фактически большинство служб работает в общем адресном пространстве ядра, поэтому с точки зрения

изоляции система ведёт себя как монолитная. Linux открыто следует монолитной модели, компенсируя её недостатки модульностью и развитыми средствами ограничения пользовательских процессов. MINIX 3 последовательно реализует принцип наименьших привилегий, но платит за это накладными расходами на передачу сообщений и меньшей производительностью.

6. Влияние размещения компонентов на изоляцию, устойчивость и безопасность

Размещение компонента в пространстве ядра означает, что он выполняется в нулевом кольце защиты и имеет доступ ко всей физической и виртуальной памяти, всем структурам ядра и оборудованию. Для такого компонента не существует внутреннего механизма, который ограничил бы последствия его ошибки или злонамеренного поведения. В связи с этим объём доверенной вычислительной базы (TCB) – совокупности кода, корректность которого необходима для безопасности системы – напрямую зависит от архитектуры ядра.

Влияние архитектурных решений на изоляцию, устойчивость и безопасность можно сформулировать следующим образом:

- изоляция: в Windows NT и Linux граница изоляции проходит только между пространством пользователя и пространством ядра, а внутри ядра драйвер сетевой карты имеет те же полномочия, что и подсистема безопасности; в MINIX 3 граница проходит вокруг каждого компонента, и драйвер ограничен своей маской привилегий;
- устойчивость: в монолитном и гибридном ядре отказ одного драйвера в большинстве случаев приводит к отказу всей системы (bugcheck или kernel panic), тогда как в MINIX 3 отказ драйвера локализуется в одном процессе и устраняется сервером реинкарнации без перезагрузки;
- безопасность: чем больше кода выполняется в режиме ядра, тем больше поверхность атаки с максимальными привилегиями; уязвимость в любом стороннем драйвере Windows или модуле Linux может быть использована для повышения привилегий до уровня ядра и обхода всех механизмов разграничения доступа;
- производительность: размещение компонентов в одном адресном пространстве исключает затраты на переключение контекста и копирование

сообщений, поэтому Windows NT и Linux обладают заметным преимуществом в скорости ввода-вывода перед MINIX 3.

Разработчики Windows и Linux частично компенсируют недостатки общего адресного пространства ядра дополнительными механизмами. В Windows применяются обязательная подпись драйверов, защита ядра от модификации PatchGuard, целостность памяти HVCI, при которой гипервизор запрещает выполнение неподписанного кода в режиме ядра, и список блокировки уязвимых драйверов [4]. В Linux используются проверка подписи модулей, режим lockdown, ограничивающий возможности даже привилегированного пользователя по изменению работающего ядра, а также средства KASLR, SMEP и SMAP [6, 7]. Эти меры снижают вероятность загрузки вредоносного кода в ядро, но не устраняют главную проблему: код, уже выполняющийся в режиме ядра, по-прежнему не изолирован.

Вынос драйверов в пространство пользователя (UMDF в Windows, FUSE, UIO и VFIO в Linux) показывает, что обе системы заимствуют идеи микроядерного подхода там, где потеря производительности допустима. Таким образом, на практике архитектуры не являются строго противоположными, а различаются долей компонентов, работающих с максимальными привилегиями.

7. Разбор ситуации компрометации драйвера

Для оценки практического значения архитектурных различий была рассмотрена следующая ситуация. Нарушитель, получивший права администратора на узле, загружает легально подписанный, но уязвимый драйвер стороннего производителя (атака типа BYOVD – «Bring Your Own Vulnerable Driver») либо использует уязвимость в уже установленном драйвере сетевого адаптера. В результате нарушитель получает возможность выполнять произвольный код с полномочиями этого драйвера.

Последствия такой компрометации для трёх свойств безопасности информации – конфиденциальности, целостности и доступности – определяются тем, какими полномочиями обладает драйвер в каждой из рассматриваемых систем. Результаты анализа приведены в таблице (Таблица 2 – Последствия компрометации драйвера для конфиденциальности, целостности и доступности).

Таблица 2 – Последствия компрометации драйвера для конфиденциальности, целостности и доступности

Свойство информации	**Windows NT**	**ОС на базе Linux**	**MINIX 3**
Конфиденциальность	Чтение любой памяти ядра и процессов, включая память lsass.exe с учётными данными (без VBS); перехват сетевого трафика и данных, проходящих через стек ввода-вывода	Чтение памяти любых процессов и ядра, ключей шифрования, перехват трафика и операций ввода-вывода; обход ограничений LSM и namespaces	Доступ только к данным, проходящим через скомпрометированный драйвер (например, трафик сетевой карты); память других процессов недоступна без выданных грантов
Целостность	Модификация структур ядра, отключение средств защиты, сокрытие процессов и файлов (руткит), подмена данных при записи на диск	Подмена таблиц и структур ядра, сокрытие процессов и модулей, отключение аудита и LSM, изменение данных файловых систем	Искажение данных, проходящих через драйвер; изменить ядро, другие серверы и драйверы невозможно из-за изоляции адресных пространств

Свойство информации	Windows NT	ОС на базе Linux	MINIX 3
Доступность	Критическая ошибка и отказ всей системы, циклическая перезагрузка, отключение служб безопасности	Kernel panic, зависание системы, отказ всех служб узла	Отказ одного устройства; сервер RS перезапускает драйвер, остальные функции системы продолжают работать

Конец Таблицы 2

Анализ показывает, что в Windows NT и Linux компрометация драйвера режима ядра равнозначна полной компрометации узла, поскольку драйвер обладает теми же полномочиями, что и само ядро. В MINIX 3 ущерб ограничен функциями конкретного драйвера, однако и здесь сохраняется риск для данных, которые через него проходят, а также риск атак через устройство с прямым доступом к памяти (DMA) при отсутствии настроенного IOMMU.

Для снижения риска компрометации драйвера предлагаются следующие меры:

1. ограничить установку драйверов только подписанными драйверами из доверенных источников и включить механизм блокировки известных уязвимых драйверов (список Microsoft, правила App Control в Windows; режим lockdown и обязательная проверка подписи модулей в Linux) [4, 7];

2. включить защиту ядра на основе виртуализации и целостность памяти (VBS и HVCI) в Windows, а также Secure Boot в Windows и Linux, чтобы предотвратить загрузку неподписанного кода в режим ядра [4];

3. минимизировать число драйверов и модулей ядра: удалить неиспользуемые, запретить автоматическую загрузку модулей (параметр kernel.modules_disabled в Linux после загрузки системы), по возможности использовать драйверы пространства пользователя (UMDF, FUSE, VFIO);

4. использовать IOMMU (Intel VT-d, AMD-Vi) для ограничения прямого доступа устройств к памяти во всех трёх системах;

5. своевременно устанавливать обновления ОС и драйверов, отслеживать бюллетени безопасности производителей оборудования;

6. ограничить круг лиц с правами администратора, поскольку загрузка драйвера требует привилегий; применять принцип наименьших привилегий для учётных записей и служб;

7. вести мониторинг событий загрузки драйверов и модулей (журналы событий Windows, системы обнаружения вторжений, контроль целостности, аудит через auditd в Linux) и анализировать дампы памяти после сбоев;

8. для систем с повышенными требованиями к надёжности рассматривать архитектуры с изоляцией драйверов (микроядра, например MINIX 3, seL4, QNX) или выносить критичные функции в отдельные виртуальные машины.

Реализация перечисленных мер не устраняет архитектурную особенность монолитного и гибридного ядра, но существенно снижает вероятность загрузки вредоносного или уязвимого кода в режим ядра и сокращает время обнаружения инцидента.

Вывод

В ходе выполнения практической работы были получены практические и теоретические навыки в сравнительном анализе архитектуры операционных систем Windows NT, Linux и MINIX 3, определении расположения их основных системных компонентов и оценке влияния архитектурных решений на изоляцию, устойчивость и информационную безопасность.

Было установлено, что Windows NT использует гибридное ядро, Linux – монолитное модульное ядро, а MINIX 3 – микроядро с многосерверной организацией. Общими для всех трёх систем являются аппаратное разделение режимов процессора, изоляция адресных пространств пользовательских процессов и предоставление приложениям интерфейса системных вызовов. Основное различие состоит в объёме кода, выполняющегося с максимальными привилегиями: в Windows NT и Linux в пространстве ядра находятся драйверы, файловые системы и сетевой стек, а в MINIX 3 эти компоненты вынесены в изолированные процессы пространства пользователя.

Разбор ситуации компрометации драйвера показал, что в Windows NT и Linux такой инцидент приводит к полной потере конфиденциальности, целостности и доступности узла, тогда как в MINIX 3 последствия ограничены функциями отдельного драйвера, а его отказ устраняется сервером реинкарнации. Вместе с тем каждая архитектура имеет ограничения: монолитные и гибридные ядра обладают большой доверенной вычислительной базой и требуют дополнительных механизмов защиты (подпись драйверов, HVCI, lockdown, Secure Boot), а микроядерная MINIX 3 уступает им в производительности, зрелости и поддержке оборудования. Выбор архитектуры, таким образом, является компромиссом между производительностью и совместимостью с одной стороны и изоляцией и устойчивостью с другой.

Список использованных источников

1. Yosifovich P., Ionescu A., Russinovich M. E., Solomon D. A. Windows Internals. Part 1: System architecture, processes, threads, memory management, and more. – 7th ed. – Redmond : Microsoft Press, 2017. – 800 p.
2. Overview of Windows Components [Электронный ресурс] // Microsoft Learn. – URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/overview-of-windows-components (дата обращения: 09.11.2026).
3. User mode and kernel mode [Электронный ресурс] // Microsoft Learn. – URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/gettingstarted/user-mode-and-kernel-mode (дата обращения: 09.11.2026).
4. Microsoft recommended driver block rules [Электронный ресурс] // Microsoft Learn. – URL: https://learn.microsoft.com/en-us/windows/security/application-security/application-control/app-control-for-business/design/microsoft-recommended-driver-block-rules (дата обращения: 09.11.2026).
5. Timeout Detection and Recovery (TDR) [Электронный ресурс] // Microsoft Learn. – URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/display/timeout-detection-and-recovery (дата обращения: 09.11.2026).
6. The Linux Kernel documentation [Электронный ресурс]. – URL: https://docs.kernel.org/ (дата обращения: 09.11.2026).
7. Kernel module signing facility [Электронный ресурс] // The Linux Kernel documentation. – URL: https://docs.kernel.org/admin-guide/module-signing.html (дата обращения: 09.11.2026).
8. Tainted kernels [Электронный ресурс] // The Linux Kernel documentation. – URL: https://docs.kernel.org/admin-guide/tainted-kernels.html (дата обращения: 09.11.2026).

9. Ubuntu 24.04 LTS (Noble Numbat) release notes [Электронный ресурс] // Ubuntu Community Hub. – URL: https://discourse.ubuntu.com/t/ubuntu-24-04-lts-noble-numbat-release-notes/39890 (дата обращения: 09.11.2026).

10. Tanenbaum A. S., Woodhull A. S. Operating Systems: Design and Implementation. – 3rd ed. – Upper Saddle River : Prentice Hall, 2006. – 1054 p.

11. Herder J. N., Bos H., Gras B., Homburg P., Tanenbaum A. S. MINIX 3: A Highly Reliable, Self-Repairing Operating System // ACM SIGOPS Operating Systems Review. – 2006. – Vol. 40, No. 3. – P. 80–89.

12. MINIX 3 [Электронный ресурс] : официальный сайт проекта. – URL: https://www.minix3.org/ (дата обращения: 09.11.2026).

13. Таненбаум Э., Бос Х. Современные операционные системы. – 4-е изд. – СПб. : Питер, 2015. – 1120 с.